Chapter 3 作业

【23336128】
【梁力航】

数据库模式

大学数据库模式（Fig.0）

- **classroom**(building, room_number, capacity)
- **department**(dept_name, building, budget)
- **course**(course_id, title, dept_name, credits)
- **instructor**(ID, name, dept_name, salary)
- **section**(course_id, sec_id, semester, year, building, room_number, time_slot_id)
- **teaches**(ID, course_id, sec_id, semester, year)
- **student**(ID, name, dept_name, tot_cred)
- **takes**(ID, course_id, sec_id, semester, year, grade)
- **advisor**(s_ID, i_ID)
- **time_slot**(time_slot_id, day, start_time, end_time)
- **prereq**(course_id, prereq_id)

银行数据库模式（Fig.1）

- **branch**(branch_name, branch_city, assets)
- **customer**(ID, customer_name, customer_street, customer_city)
- **loan**(loan_number, branch_name, amount)
- **borrower**(ID, loan_number)
- **account**(account_number, branch_name, balance)
- **depositor**(ID, account_number)

问题1：使用大学模式编写SQL查询

a) 查找至少选修过一门计算机科学课程的每位学生的ID和姓名，确保结果中没有重复的姓名

```sql
SELECT DISTINCT student.ID, student.name
FROM student
JOIN takes ON student.ID = takes.ID
JOIN course ON takes.course_id = course.course_id
WHERE course.dept_name = 'Comp. Sci.';
```

b) 查找未选修过2017年之前开设的任何课程的每位学生的ID和姓名

```
SELECT ID, name
FROM student
WHERE ID NOT IN (
    SELECT ID
    FROM takes
    WHERE year < 2017
);
```

或使用NOT EXISTS：

```
SELECT ID, name
FROM student
WHERE NOT EXISTS (
    SELECT *
    FROM takes
    WHERE takes.ID = student.ID
      AND takes.year < 2017
);
```

c) 找到每个系（department）的教师的最高薪水

```
SELECT dept_name, MAX(salary) AS max_salary
FROM instructor
GROUP BY dept_name;
```

d) 找到在所有系（departments）中计算出的每个系最高薪水的最低值

```
SELECT MIN(max_salary) AS min_of_max_salary
FROM (
    SELECT dept_name, MAX(salary) AS max_salary
    FROM instructor
    GROUP BY dept_name
) AS dept_max_salaries;
```

问题2：重写查询（不使用WITH子句）

原查询：

```
WITH dept_total(dept_name, value) AS
    (SELECT dept_name, SUM(salary)
     FROM instructor
     GROUP BY dept_name),
dept_total_avg(value) AS
    (SELECT AVG(value)
     FROM dept_total)
SELECT dept_name
FROM dept_total, dept_total_avg
WHERE dept_total.value >= dept_total_avg.value;
```

重写后（不使用WITH）：

```
SELECT dept_name
FROM (
    SELECT dept_name, SUM(salary) AS value
    FROM instructor
    GROUP BY dept_name
) AS dept_total
WHERE value >= (
    SELECT AVG(value)
    FROM (
        SELECT dept_name, SUM(salary) AS value
        FROM instructor
        GROUP BY dept_name
    ) AS dept_total_inner
);
```

问题3：重写WHERE语句（不使用UNIQUE）

原语句：

```
WHERE UNIQUE(SELECT title FROM course)
```

重写后：

```
WHERE (SELECT COUNT(DISTINCT title) FROM course) = (SELECT COUNT(*) FROM course)
```

或者：

```
WHERE NOT EXISTS (
    SELECT title
    FROM course c1
    WHERE EXISTS (
        SELECT *
        FROM course c2
        WHERE c1.title = c2.title
          AND c1.course_id <> c2.course_id
    )
)
```

问题4：构建针对银行数据库的SQL查询

a) 查找银行中的拥有账户但没有贷款的每位顾客的ID

```
SELECT DISTINCT ID
FROM depositor
WHERE ID NOT IN (
    SELECT ID
    FROM borrower
);
```

或使用EXCEPT：

```
SELECT ID
FROM depositor
EXCEPT
SELECT ID
FROM borrower;
```

b) 查找与顾客'12345'住在相同街道和城市的顾客的ID

```
SELECT c1.ID
FROM customer c1
JOIN customer c2 ON c1.customer_street = c2.customer_street
                 AND c1.customer_city = c2.customer_city
WHERE c2.ID = '12345'
  AND c1.ID <> '12345';
```

或使用子查询：

```
SELECT ID
FROM customer
WHERE customer_street = (SELECT customer_street FROM customer WHERE ID = '12345')
  AND customer_city = (SELECT customer_city FROM customer WHERE ID = '12345')
  AND ID <> '12345';
```

c) 查找每个至少有一位在银行拥有账户并住在"Harrison"的顾客的分行的名称

```
SELECT DISTINCT branch_name
FROM account
WHERE account_number IN (
    SELECT account_number
    FROM depositor
    JOIN customer ON depositor.ID = customer.ID
    WHERE customer.customer_city = 'Harrison'
);
```

或使用JOIN：

```sql
SELECT DISTINCT account.branch_name
FROM account
JOIN depositor ON account.account_number = depositor.account_number
JOIN customer ON depositor.ID = customer.ID
WHERE customer.customer_city = 'Harrison';
```

问题5：构建针对银行数据库的SQL查询

a) 找到每位在"Brooklyn"的每个分行都有账户的顾客

```sql
SELECT DISTINCT d.ID
FROM depositor d
WHERE NOT EXISTS (
    SELECT branch_name
    FROM branch
    WHERE branch_city = 'Brooklyn'
      AND branch_name NOT IN (
          SELECT a.branch_name
          FROM account a
          JOIN depositor d2 ON a.account_number = d2.account_number
          WHERE d2.ID = d.ID
      )
);
```

或使用除法（关系代数）的等价形式：

```sql
SELECT d.ID
FROM depositor d
WHERE NOT EXISTS (
    SELECT b.branch_name
    FROM branch b
    WHERE b.branch_city = 'Brooklyn'
      AND NOT EXISTS (
          SELECT *
          FROM account a
          JOIN depositor d2 ON a.account_number = d2.account_number
          WHERE d2.ID = d.ID
            AND a.branch_name = b.branch_name
      )
);
```

b) 找到银行中所有贷款金额的总和

```sql
SELECT SUM(amount) AS total_loan_amount
FROM loan;
```

c) 找到具有资产超过至少一个位于"Brooklyn"分行的资产的所有分行名称

```
SELECT branch_name
FROM branch
WHERE assets > SOME (
    SELECT assets
    FROM branch
    WHERE branch_city = 'Brooklyn'
);
```

或使用ANY：

```
SELECT branch_name
FROM branch
WHERE assets > ANY (
    SELECT assets
    FROM branch
    WHERE branch_city = 'Brooklyn'
);
```